

BUSINESS REQUIREMENTS DOCUMENT

CarSage

An All-in-One Car Companion App

Prepared by: **Mohamad Dib**

Version: **1.0**

Date: **September 2026**

Status: **Approved for Development — MVP Scope**

1. Document Control

1.1 Document Information

Field	Detail
Project Title	CarSage — An All-in-One Car Companion App
Prepared By	Mohamad Dib
Document Type	Business Requirements Document (BRD)
Version	1.0
Date	September 2026
Approval Status	Scope approved by project mentor — narrowed to 2 core features

1.2 Revision History

Version	Date	Author	Description of Change
0.1	August 2026	Mohamad Dib	Initial brainstorm and feature list drafted
0.2	September 2026	Mohamad Dib	Scope narrowed to 2 core features per mentor feedback
1.0	September 2026	Mohamad Dib	Final BRD compiled and approved for development

1.3 Distribution List

Name	Role	Purpose
Mohamad Dib	Project Owner / Developer	Author and implementer of the project
Mohammad Dawi	Project Mentor / Reviewer	Reviews scope, feasibility, and code quality

2. Executive Summary

CarSage is a web-based companion application designed to simplify car ownership for everyday drivers. Rather than juggling separate tools for navigation, fuel cost estimation, and troubleshooting car issues, users will have a single platform to manage their car profile, plan trips with accurate cost estimates, and receive AI-guided advice on whether a car problem can be fixed independently or requires professional service.

Following an initial brainstorm covering four potential features, the project mentor advised narrowing the scope to two core, non-basic features to ensure depth, technical quality, and a strong demonstration of applied engineering skills. The approved MVP scope consists of two foundational features (Car Onboarding and Car Profile) and two core features (Trip Planner and AI Advisor), built on a modern stack of React, Supabase, and FastAPI. This document formalizes the business requirements, technical approach, and success criteria for that approved scope.

3. Project Overview

3.1 Project Brief

CarSage is a companion app for everyday car owners. It combines the two capabilities car owners need most often into a single platform: a trip cost planner that estimates fuel cost and travel time using real-time traffic and local gas prices,

and an AI advisor that helps diagnose car issues and tells the user whether it is a DIY fix or requires a mechanic. The goal is to replace the scattered mix of notes apps, maps, and online forums that car owners currently rely on with one consolidated, easy-to-use tool.

3.2 Problem Statement

- Car owners track maintenance and car details across notes apps, spreadsheets, or memory, with no central record.
- Trip cost estimation (fuel + traffic) requires manually combining data from maps apps and gas price sources.
- When a car issue arises, owners often do not know whether it is safe/simple to fix themselves or requires professional service, leading to either unnecessary shop visits or risky DIY attempts.

3.3 Project Goals & Objectives

1. Deliver a working web application (MVP) demonstrating the two approved core features.
2. Validate user interest through a landing page / demo before committing to a full mobile build.
3. Build a technically sound foundation (Supabase + FastAPI + React) that can scale to a mobile app (React Native) later.
4. Demonstrate strong, well-structured code quality as the primary academic objective of the project.

3.4 Target Users

Everyday car owners who want a simple way to manage their vehicle information, estimate trip costs, and get quick, reliable guidance on car issues — without needing to be car enthusiasts or technical experts.

3.5 Stakeholders

Stakeholder	Interest / Role
Mohamad Dib (Developer)	Designs, builds, and delivers the application; owns technical decisions
Mohammad Dawi (Project Mentor)	Reviews scope and code quality; approves milestones and final submission
End Users (Car Owners)	Primary users of the deployed application; source of validation feedback

4. Scope

4.1 In Scope (MVP)

Per mentor guidance, the project focuses on two core (non-basic) features, built on top of two foundational (basic) features, to ensure code quality and depth over breadth.

Category	Feature	Description
Basic	Car Onboarding	Add a car by manual entry, or scan the registration card as a shortcut to auto-fill specs.
Basic	Car Profile	View and manage the specs and details of the user's saved car(s).
Core	Trip Planner	Enter a destination; the app estimates fuel cost using local gas prices and travel time adjusted for real-time traffic.
Core	AI Advisor	User describes a car issue or asks a question; the AI determines whether it is DIY-fixable (with guidance) or requires a mechanic.

4.2 Out of Scope (Future Enhancements)

- Maintenance Log (service history tracking with reminders) — original brainstorm feature; deprioritized per mentor guidance to keep MVP scope to 2 core features. To be added if time allows after core features are complete.
- OCR-based automatic parsing of scanned registration/car cards during onboarding — original brainstorm feature; manual entry (with scanning as a basic shortcut) is the MVP fallback, full automatic field extraction is a stretch goal.
- Extended car profile content (fun facts/trivia about the specific car beyond core specs) — from the original brainstorm; deferred beyond MVP.
- Native mobile app (React Native) — the original project vision was a mobile app; per plan, the website is being built and validated first, with the mobile app as a later phase if approved.
- Multi-car fleet management, cost-sharing, or social features — natural extensions considered during planning; not part of the original brainstorm or MVP.
- Waitlist/interest-validation tooling beyond a simple landing page (e.g., surveys, analytics dashboards) — the MVP website itself serves as the validation vehicle; deeper validation tooling is a future consideration.

5. User Personas

Persona	Profile	Needs
Sarah, 34 — Daily Commuter	Owns one car, commutes to work daily, has limited car maintenance knowledge.	Wants to know trip costs before a long drive and needs simple guidance when something feels wrong with the car.
Karim, 45 — Family Driver	Owns the family car, occasionally handles small repairs himself to save money.	Wants a quick, trustworthy way to check if a car issue is safe to fix at home before calling a mechanic.
Lea, 27 — New Driver	Recently got her own car, unfamiliar with car specs and terminology.	Wants an easy onboarding process and clear, jargon-free explanations of car issues.

6. Use Cases & User Stories

ID	User Story	Related Feature
US-01	As a new user, I want to add my car by filling a simple form, so that I can start using the app quickly.	Car Onboarding
US-02	As a user, I want to scan my registration card, so that I do not have to type in all my car's details manually.	Car Onboarding
US-03	As a user, I want to view and edit my car's saved details, so that my information stays accurate.	Car Profile
US-04	As a driver, I want to enter a destination and see the estimated fuel cost, so that I can budget for my trip.	Trip Planner
US-05	As a driver, I want to see traffic-adjusted travel time, so that I know how long my trip will realistically take.	Trip Planner
US-06	As a car owner, I want to describe a strange noise or warning light, so that I know if I can fix it myself.	AI Advisor
US-07	As a car owner, I want step-by-step DIY guidance when a fix is simple, so that I can save money on minor issues.	AI Advisor
US-08	As a car owner, I want to be clearly told when an issue requires a mechanic, so that I avoid worsening a serious problem.	AI Advisor

7. Functional Requirements

7.1 Car Onboarding (Basic)

Req. ID	Requirement
FR-01	User can create a car profile via a manual entry form (make, model, year, engine type, etc.).
FR-02	User can optionally scan the car's registration card as a shortcut to speed up entry.
FR-03	Car data is validated on submission and saved to the user's account, persisting across sessions.

7.2 Car Profile (Basic)

Req. ID	Requirement
FR-04	System displays all saved specs and details for the user's car in a dedicated profile view.
FR-05	User can edit or update car information at any time after onboarding.

7.3 Trip Planner (Core Feature 1)

Req. ID	Requirement
FR-06	User can enter a destination (and optionally a starting point) for a planned trip.
FR-07	System retrieves route, distance, and real-time traffic data from a mapping service.
FR-08	System retrieves local fuel prices and calculates estimated fuel cost using the car's known fuel efficiency.
FR-09	System displays estimated cost and estimated travel time to the user before the trip begins.

7.4 AI Advisor (Core Feature 2)

Req. ID	Requirement
FR-10	User can describe a car issue in natural language (symptoms, noises, warning lights, etc.).
FR-11	AI evaluates the issue using car context (make/model/year) and returns a DIY-or-mechanic recommendation.
FR-12	When DIY-fixable, the system provides clear, step-by-step guidance.
FR-13	User can ask general car-related questions and receive relevant, contextual answers.

8. Non-Functional Requirements

Category	Requirement
Performance	Trip cost estimates and AI Advisor responses should return within a few seconds under normal network conditions.
Usability	Interface must be simple and jargon-free, usable by non-technical, everyday car owners.
Security	User accounts and car data must be protected via Supabase Auth with row-level security policies.
Reliability	Core features must degrade gracefully (clear error messages) if an external API (Maps, fuel prices, LLM) is unavailable.
Scalability	Architecture must support adding features (e.g., Maintenance Log) and migrating to a mobile app without a full rebuild.
Maintainability	Codebase must be well-structured and documented to reflect strong software engineering practices.

9. AI Capabilities

AI Capability	Purpose	MVP Priority
DIY vs. Mechanic Advisor	Core differentiator — assesses car issues and recommends DIY guidance or professional service	Required
Car Q&A Assistant	Answers general car and maintenance questions from the user	Required
Registration OCR Parsing	Auto-fills car specs from a scanned registration card photo	Stretch goal

10. Technical Architecture

10.1 Technology Stack

Layer	Technology	Purpose
Frontend	React (web)	User interface; foundation reusable for React Native mobile app later
Backend / Database	Supabase (PostgreSQL, Auth, Storage)	User accounts, car profiles, and data persistence
AI / Logic Service	FastAPI (Python)	Hosts AI Advisor logic and trip cost calculations
AI Model	LLM API (e.g., Claude)	Powers the DIY vs. mechanic advisor and Q&A assistant
Maps / Traffic	Google Maps API (Directions & Distance Matrix)	Route, distance, and real-time traffic data for Trip Planner
Fuel Prices	Region-specific gas price data source (TBD)	Local fuel price input for trip cost calculation

10.2 High-Level Architecture

- React frontend communicates directly with Supabase for standard CRUD operations: authentication, car profiles, and stored data.
- React frontend communicates with the FastAPI service for AI-driven and computation-heavy operations: the AI Advisor and Trip Planner cost calculations.
- FastAPI reads/writes to the Supabase Postgres database when persisting AI-processed results (e.g., saved trip estimates or advisor conversations).
- FastAPI integrates with external APIs (LLM provider, Google Maps, fuel price source) to fulfil requests.

10.3 Data Flow Summary

1. User interacts with the React frontend (web browser).
2. Authentication and simple data operations (car profile CRUD) go directly to Supabase.
3. Trip Planner and AI Advisor requests are routed to the FastAPI service.
4. FastAPI calls external APIs (Google Maps, fuel price source, LLM provider) as needed.
5. FastAPI returns results to the frontend and optionally persists them to Supabase for history.

Note: The full Entity Relationship Diagram (ERD), including table definitions and key relationships, is provided as a separate companion document (CarSage_ERD).

11. Assumptions & Constraints

11.1 Assumptions

- Users have a registered account (via Supabase Auth) before accessing car features.
- A working internet connection is required for Trip Planner and AI Advisor features (both depend on external APIs).
- A reliable regional fuel price data source is available or can be reasonably approximated.

11.2 Constraints

- Project scope limited to 2 core features plus basics, per mentor guidance, to prioritize code quality.
- MVP is delivered as a website first; native mobile app is a future phase, not part of this deliverable.
- AI Advisor recommendations are guidance only and not a substitute for professional certified mechanic diagnosis.

12. Risks & Mitigations

Risk	Impact	Mitigation
External API limits or downtime (Maps, fuel prices, LLM)	Trip Planner or AI Advisor may fail to return results	Implement graceful error handling and, where possible, cached/fallback data
Inaccurate or unavailable regional fuel price data	Trip cost estimates may be imprecise	Research and select the most reliable available source; clearly label estimates as approximate
AI Advisor gives incorrect DIY guidance	User safety risk if a serious issue is misclassified as DIY-fixable	Bias recommendations toward caution (“see a mechanic”) when confidence is low; add a visible disclaimer
Scope creep back toward original 4-feature idea	Reduced code quality due to divided focus	Maintain strict adherence to mentor-approved 2-core-feature scope; log extras as future enhancements only

13. Success Criteria

1. User can create an account, add a car, and view/edit its profile without errors.
2. User can enter a destination and receive an accurate estimated fuel cost and traffic-adjusted travel time.
3. User can describe a car issue and receive a clear DIY-or-mechanic recommendation from the AI Advisor.
4. Core features function reliably end-to-end (frontend → FastAPI/Supabase → external APIs → response).
5. Mentor approval obtained on final implementation, confirming code quality expectations are met.

14. Project Timeline & Milestones

Phase	Milestone	Target Outcome
Phase 1	Requirements & Design	BRD finalized (this document); UI wireframes for onboarding, profile, trip planner, and advisor screens
Phase 2	Foundation Setup	Supabase project, database schema, and auth configured; React app scaffolded
Phase 3	Basic Features	Car Onboarding and Car Profile implemented and tested

Phase	Milestone	Target Outcome
Phase 4	Trip Planner	Google Maps integration and fuel cost calculation implemented via FastAPI
Phase 5	AI Advisor	LLM integration for DIY-vs-mechanic recommendations implemented via FastAPI
Phase 6	Testing & Polish	End-to-end testing, bug fixes, and UI refinement
Phase 7	Submission	Final website demo delivered to mentor for review

15. Glossary

Term	Definition
BRD	Business Requirements Document — this document, defining project scope and requirements.
MVP	Minimum Viable Product — the smallest functional version of the app that delivers core value.
Supabase	Backend-as-a-service platform providing a Postgres database, authentication, and storage.
FastAPI	A Python web framework used to build the AI/logic service for this project.
LLM	Large Language Model — the AI model powering the AI Advisor's recommendations.
DIY	Do It Yourself — a repair or task the user can safely complete without a professional.
OCR	Optical Character Recognition — technology to extract text from images (e.g., registration cards).

16. Notes

- Maintenance Log was part of the original brainstorm but was deprioritized per mentor feedback to keep scope focused; it remains a planned enhancement if time permits.
- Differentiation strategy: most existing car apps (Fueelly, Drivvo, aCar) cover logging but lack an integrated AI diagnostic advisor — this is CarSage's key differentiator.
- The website serves as both the MVP deliverable and a validation tool (interest/waitlist) ahead of a potential future native app build.

17. Appendix — Acceptance Test Checklist

The following checklist will be used to validate that each requirement has been correctly implemented before final submission.

Req. ID	Test Scenario	Expected Result
FR-01	User fills out and submits the manual car entry form.	Car profile is created and saved to the user's account.
FR-02	User uploads/scans a registration card image.	Relevant fields are pre-filled (or a clear fallback to manual entry is shown).
FR-04 / FR-05	User navigates to their car profile, edits a field, and saves.	Specs display accurately and updates persist after page reload.

Req. ID	Test Scenario	Expected Result
FR-06	User enters a destination in the Trip Planner.	System accepts input and initiates a route/cost lookup.
FR-08 / FR-09	Trip Planner calculation completes for a valid route.	Estimated fuel cost and traffic-adjusted travel time are both displayed.
FR-10	User submits a car issue description to the AI Advisor.	System returns a relevant, car-context-aware response.
FR-11 / FR-12	AI Advisor processes an issue description.	Response clearly states DIY-fixable (with guidance) or requires a mechanic.

17.1 Sign-Off

This document, including all scope, requirements, and success criteria described above, has been reviewed and approved by the project mentor. Any changes to scope after this point should be documented as a revised version of this BRD.

Role	Name	Date
Project Owner	Mohamad Dib	September 2026
Project Mentor	Mohammad Dawi	_____